

RAPPORT

Gruppenavn og kandidater: Gruppe 5

Anastasia Katanova, anastasiak@uia.no

Alexander Glasdam Andersen, alexandega@uia.no

Frank Hovet, frankh@uia.no

Tobias Molland, tobiasmol@uia.no

Lars Kydland, larsky@uia.no

Simen Emil Wiig Holmen, seholmen@uia.no

Emnekode	IS-218
Emnenavn	Geografiske Informasjonssystemer, AI og IoT; Introduksjon og Anvendelse
Emneansvarlig	Geir Inge Hausvik
Innleveringsfrist	16. mai 2025 14.00
Antall sider (inkl. forside)	18
Tittel på prosjekt/produkt	Oppgave 5: Semesterprosjekt

Vi bekrefter at vi ikke siterer eller på annen måte bruker andres arbeid uten at dette er oppgitt og at alle referanser er oppgitt i litteraturlisten.	Ja x	Nei
--	-----------------------	------------

Kan besvarelsen brukes til undervisningsformål?	Ja x	Nei
---	-----------------------	------------

Vi bekrefter at alle i gruppa har bidratt til besvarelsen	Ja x	Nei
---	-----------------------	------------

Innholdsfortegnelse

Innholdsfortegnelse.....	2
Figurliste.....	2
1.0 Problemstilling.....	3
2.0 Arbeidet.....	3
2.1 utfordringer.....	3
2.2 Kompleksitet.....	4
2.3 Kvalitet.....	5
3.0 Beskrivelse av løsning.....	7
3.1 Datagrunnlaget.....	7
3.2 Funksjoner.....	11
4.0 Diskusjon.....	15
4.1 Beslutninger.....	15
4.2 Hva har vi lært.....	16
5.0 Veien videre.....	17
Referanser.....	18

Figurliste

Figur 1: Antall rader i “linesForShortestPath” tabellen.....	4
Figur 2: Kartprojeksjonen til linjene i “linesForShortestPath” tabellenskriv.....	5
Figur 3: Spørring mot OverPassTurbo-API.....	8
Figur 4: Funksjon for Vei-distanseberegning.....	8
Figur 5: Parametre for trening av KI-modell.....	9
Figur 6: Alle bygninger markert av KI.....	10
Figur 7: Alle områder ikke markert av KI delt opp i mindre rektangler.....	10
Figur 8: Linjer mellom alle punkt som ikke krysser umarkert område.....	11
Figur 9: Henting av data fra databasen.....	12
Figur 10: Simplifisert fetch_lines funksjon for å vise logikk.....	13
Figur 11: Simplifisert shortestpath funksjon for å vise logikk.....	15

1.0 Problemstilling

Det er ikke alltid klart hvor godt skoler og barnehager er dekket av beredskapstjenester som politi, brann og helse (Utdanningsdirektoratet, u.å.). Dette er spesielt tilfellet i kritesituasjoner hvor normal infrastruktur kan være utilgjengelig (Nasjonal sikkerhetsmyndighet, 2022).

I en kritesituasjon er rask og sikker tilgang til beredskapstjenester avgjørende for å redde liv. Identifisering av svak dekning og forbedring av rutevalg kan styrke kriseberedskap og redusere risiko for sårbare grupper som barn i barnehager og skoler.

Løsningen i dette prosjektet er utvikling av en karttjeneste som beregner korteste vei til nærmeste beredskapssenter, enten fra nåværende lokasjon eller en spesifisert skole eller barnehage. Tjenesten kan også bruke bildegjenkjenning og kunstig intelligens til å oppdage alternative ruter, som snarveier utenom veinett, for å forbedre evakueringsmuligheter. En slik løsning vil kunne inkludere mulighet for å unngå blokkeringer slik som for eksempel veisperringer eller flom.

2.0 Arbeidet

Løsningen ligger på <https://github.com/simholmen/IS218-gruppe5-oppgave5> og en video som forklarer prosjektet ligger på <https://youtu.be/qYw4H6qkzm0>.

2.1 Utfordringer

Et av formålene med applikasjonen er å finne korteste ruten uten at dette nødvendigvis tar hensyn til eksisterende veier. Dette vil bety at en av hovedutfordringene blir å utforme en måte applikasjonen kan danne en vei mellom destinasjoner, men likevel ta stilling til hindringer som kan oppstå i veien mellom disse to punktene. En naturlig løsning til dette var å generere et nettverk av veier over hele området applikasjonen skal ta stilling til. En annen mulig fremgangsmåte ville vært å bruke et raskt ekspanderende tilfeldig tree (*Rapidly-exploring Random tree, RRT*) for å lage veier, men dette er saktere enn et navigasjons kart gitt at navigasjonsnettet er generert på forhånd.

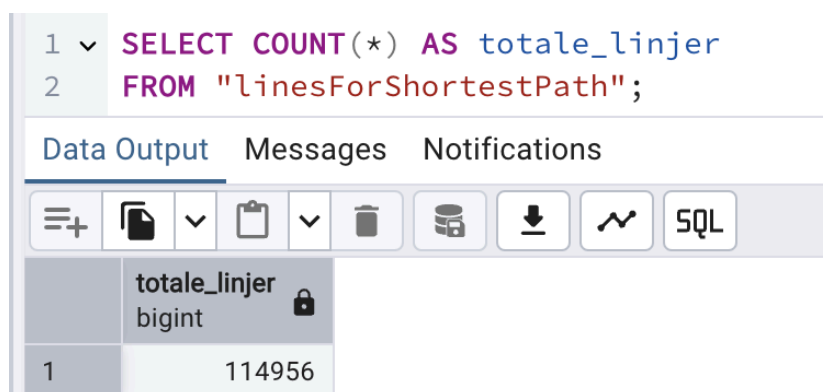
En annen gjengående utfordring, som er mer generell for faget, gikk ut på hvordan kartapplikasjonen skulle kunne nøyaktig gjengi virkeligheten. Denne problemstillingen hadde

to hovedkomponenter, en teknisk komponent, som hovedsakelig gikk ut på valg av kart, og et generaliseringsproblem, som gikk ut på at enkelte detaljer som er viktige for slutt-løsninger kan abstraheres bort ved kart.

2.2 Komplexitet

For å kunne implementere funksjonene som svarer på den nye problemstillingen, krevde det at løsningen måtte bli mer kompleks. Grunnen til at løsningen måtte bli mer kompleks var at funksjonene som måtte implementeres håndterte svært mye mer data. Dette førte til at det ikke var gunstig å håndtere alt data gjennom frontend. Funksjonene måtte også kjøres i python, noe som betydde at gruppen utviklet en backend, for å kunne håndtere den store mengden data og kjøre funksjoner for å løse problemstillingen.

For å kunne kjøre backend i løsningen, må den bli kjørt gjennom en Python fortolker (*python interpreter*). Det finnes mange forskjellige måter å gjøre dette på. Måten gruppen har løst dette på er ved å bruke QGIS sin innebygde interpreter. Ved å bruke denne måten å fortolke python på er det lettere å kjøre de innebygde funksjonene fra QGIS som er essensielle i løsningen. Årsaken til at denne fremgangsmåten ble prioritert var fordi QGIS gikk ofte igjen i undervisningen, og gruppen anså derfor det som relevant å bruke applikasjonen aktivt i løsningen.



The screenshot shows the QGIS SQL console interface. At the top, there is a text area with a SQL query: `SELECT COUNT(*) AS totale_linjer FROM "linesForShortestPath";`. Below the text area are three tabs: 'Data Output', 'Messages', and 'Notifications'. The 'Data Output' tab is active, showing a table with one column named 'totale_linjer' of type 'bigint'. The table contains one row with the value '114956'.

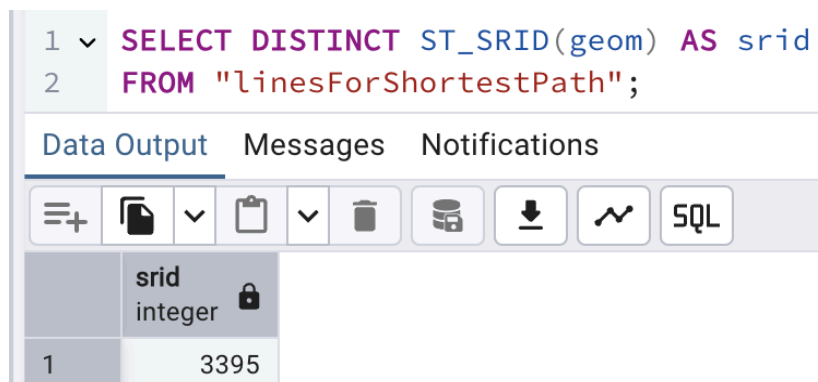
	totale_linjer bigint
1	114956

Figur 1: Antall rader i "linesForShortestPath" tabellen

Dataen som blir brukt for å visualisere løsningen må også bli lagret og hentet for å kunne bli behandlet eller vist på kartet. Som vist i figur 1, inneholder tabellen over 110.000 linjer, noe

som betyr at den store mengden data må bli behandlet på rett måte. Forskjellen på måten punktene for brannstasjoner, sykehus og politistasjoner og linjene i “linesForShortestPath” blir behandlet på er at punktene blir hentet direkte gjennom frontend, mens linjene blir hentet gjennom backend. Grunnen til at dette er at det er mer effektivt å hente data gjennom backend enn mellom frontend gitt store mengder data. Kallet som henter data gjennom frontend har også en innebygd begrensing, hvor den bare kan hente opp til 1000 objekter fra Supabase (Supabase, u.å.). Denne begrensingen gjør funksjonen egnet til å kun hente data fra tabeller med mindre enn 1000 objekter. Større datahentinger bør derfor delegeres til backend-servere.

En annen viktig teknisk kompleksitet i løsningen er kartprojeksjoner. Kartet i Leaflet bruker EPSG: 4326 i WGS 84 projeksjonen sammen med de fleste andre GPS systemer (Jurkowska, 2023). Det betyr at alle punkter som blir markert i kartet på leaflet vil være i EPSG: 4326, noe som er en annen projeksjon enn mye av dataen er lagret i. For eksempel er linjene i “linesForShortestPath” lagret i EPSG: 3395 som vist i figur 2. Dette betyr at for at punktene som blir satt i leaflet skal kunne samhandle med linjene i “linesForShortestPath”, må de bli transformert til en felles projeksjon.



```
1 SELECT DISTINCT ST_SRID(geom) AS srid
2 FROM "linesForShortestPath";
```

Data Output	
	srid integer
1	3395

Figur 2: Kartprojeksjonen til linjene i “linesForShortestPath” tabellenskriv

2.3 Kvalitet

Kvalitet i dette prosjektet har vært ivaretatt gjennom flere tiltak knyttet til datagrunnlag, teknisk implementasjon, brukertesting og ytelse.

Datasettene er hentet fra offentlige og anerkjente kilder. Det ble gjennomført manuell gjennomgang av datasettene for å sikre at nødetatpunktene var reelle og relevante. Dette var viktig for å oppnå høy nøyaktighet i karttjenesten, særlig i kritiske situasjoner der brukere er avhengige av presise og pålitelige ruter.

Ytelse og effektiv databehandling har også vært et kriterium. I konteksten av at systemet håndterer over 110 000 linjer i rutenettverket ble det gjort et teknisk valg om å hente denne datamengden via backend. Punktdata, som brannstasjoner og sykehus, ble i stedet hentet direkte i frontend. Dette reduserte belastningen på nettleseren og førte til raskere respons, noe som forbedrer brukeropplevelsen og sikrer et mer robust system ved bruk i sanntid (HeavyAI, 2026). Bruk av EPSG:3395 for kalkulering i backend bidro til mer nøyaktige kalkulasjoner, til tross for at EPSG:4326 (WGS 84) brukes i frontend. Kvaliteten ble også validert gjennom ekstern tilbakemelding og fysisk testing. Applikasjonen ble presentert på en messe, hvor det kom tilbakemeldinger fra medstudenter og veiledere. Det ble foreslått forbedringer knyttet til blant annet datagrunnlag og visuell fremstilling, som det ble tatt hensyn til i etterkant. I tillegg ble det gjennomført en fysisk test hvor en foreslått rute ble valgt og gått for å undersøke om rutevalget var realistisk i praksis. En utfordring som kom opp i testen var at deler av ruten gikk gjennom privat eiendom, noe som i realiteten kan være problematisk. Ellers ble ruten gjennomgått og fullført med suksess, i konklusjon bekrefter testen at rutevalg var realistisk.

Et annet viktig aspekt ved kvalitet har vært bruken av kunstig intelligens for å generere alternative ruter. Modellen ble trent opp på flyfoto og benyttet til å identifisere bygninger og dermed navigerbare områder i terreng uten vei. Det ble imidlertid observert at ikke alle bygninger ble korrekt gjenkjent, noe som representerer en utfordring for kvaliteten i visse scenarier. Likevel bidrar bruken av AI til et mer fleksibelt rutenett og åpner for videre forbedring av løsningen.

Samlet sett har prosjektet hatt en helhetlig tilnærming til kvalitet ved å kombinere pålitelig datagrunnlag, optimalisering av ytelse, teknisk nøyaktighet, tilbakemeldinger fra brukere og fysisk validering. Dette har vært avgjørende for å sikre at løsningen fungerer etter hensikten i krisesituasjoner, og at den oppleves som pålitelig og responsiv i bruk.

3.0 Beskrivelse av løsning

Løsningen er utviklet som en nettbasert kartapplikasjon for beredskapsanalyse, med fokus på å måle og visualisere tilgjengeligheten til kritiske beredskapstjenester som brannstasjoner, politistasjoner og sykehus. Arkitekturen følger en klient-servermodell hvor frontend, backend- og databasetjenestene har hver sin oppgave som utfyller hverandre. En viktig komponent i løsningen er integrasjonen av kunstig intelligens for å identifisere alternative ruter i krisesituasjoner. I motsetning til tradisjonelle navigeringssystemer, som er begrenset til veinettverk, så bruker vår implementasjon ikke-konvensjonelle ruter i alle typer terreng som parkområder, plener og jorder.

Fra en arkitekturisk vinkel er løsningen bygget opp av flere ulike komponenter. Frontend bruker en React-basert webapplikasjon med Leaflet for kartbiblioteket. Backend bruker en Python Flask-server som integrerer ulike QGIS funksjonaliteter for mer avanserte geografiske funksjoner. Databasen tar i bruk en PostgreSQL/PostGIS-database for lagring av geografiske data implementert gjennom Supabase. I tillegg ble det brukt OpenRouteServices API for veiberegningsanalysen.

3.1 Datagrunnlaget

Løsningen bruker en hybrid tilnærming til de ulike geografiske beregningene. De enkle beregningene som luftlinjeavstand utføres direkte i frontend. Komplekse beregninger som veibasert ruting og korteste-veianalyse delegeres til egne backend-servere.

Systemet bruker tre primære datasett for lokasjonen til de ulike nødetatene, brannstasjoner, sykehus og politistasjoner. Brannstasjoner er hentet fra GeoNorge. Politistasjoner og sykehus er hentet ut ved hjelp fra OverPassTurbo API med følgende spørring:

```
[out:json][timeout:25];
// Finn kommuneområdet for Kristiansand
area["name"="Kristiansand"]["boundary"="administrative"]["admin_level"="8"]->.searchArea
;

// Hent politistasjoner og sykehus
(
  node["amenity"="police"](area.searchArea);
  way["amenity"="police"](area.searchArea);
```

```

relation["amenity"="police"](area.searchArea);

node["amenity"="hospital"](area.searchArea);
way["amenity"="hospital"](area.searchArea);
relation["amenity"="hospital"](area.searchArea);
);
out center;

```

Figur 3: Spørring mot OverPassTurbo-API

Vei-distanseberegningen kalkulerer den faktiske kjøreruten langs allerede eksisterende veinettverk. Implementasjonen av denne bruker OpenRouteService API som er basert på OpenStreetMap-data, denne tar hensyn til ulike veikategorier som motorvei, fylkesveier, kommunalveier, og fartsbegrensninger og estimerte kjøretider.

```

async function findClosestByRoad(userCoords, items) {
  // Pre-filtering for å unngå unødvendige API-kall
  const closeItems = items.filter(item => {
    // Beregn først luftlinjeavstand...
    return airDistance < MAX_AIR_DISTANCE;
  });

  // For hvert punkt, beregn veidistance via API...
  for (const item of closeItems) {
    // Kall OpenRouteService API...
    // Lagre korteste avstand og rute...
  }

  return { closestMarker, shortestDistance };
}

```

Figur 4: Funksjon for Vei-distanseberegning

I motsetning til vei-distanseberegning krever funksjonen for å identifisere alternative ruter i en krisesituasjon et eget datagrunnlag. Datagrunnlaget ble laget ved å bruke KI-modellen fra KartAI prosjektet. KI-modellen ble trent opp med bildegjenkjenning av bygninger fra flyfoto. Trening ble utført på flyfoto fra Skøyen i Oslo, som var området som ga de mest nøyaktige resultatene og høyeste IoU-verdiene for flyfoto i Kristiansand. Koden i figur 5 viser parametrene som ble brukt for å trene KI-modellen.

```

model_name = "linesForShortestPath"
model_architecture = "unet"

train_args = {
  "features": 32,

```



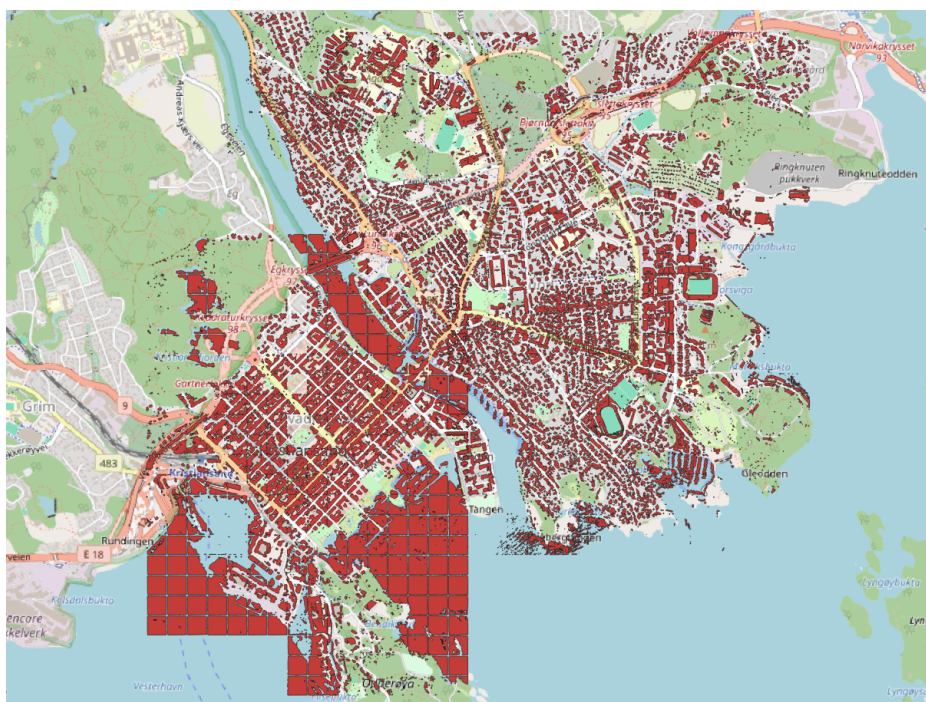
```

"depth": 4,
"optimizer": "RMSprop",
"batch_size": 8,
"model": model_architecture,
"loss": "binary_crossentropy",
"activation": "relu",
"epochs": 15
}

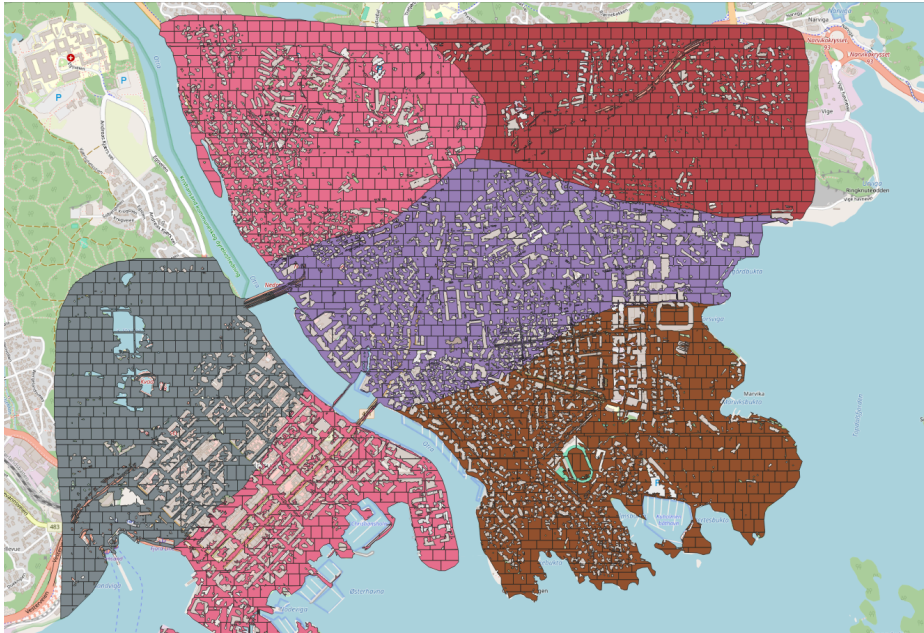
```

Figur 5: Parametre for trening av KI-modell

Etter at KI-modellen hadde blitt trent til å gi gode nok resultater, ble dataen behandlet i QGIS for å lage et datagrunnlag. Alle bygningene ble markert i kartet som vist i figur 6. Etterpå ble de omvendt slik at alt område som ikke var markert av KI ble markert, og delt opp i mindre rektangler som vist i figur 7. Her ble også alle umarkerte områder som var mindre enn en kvadratmeter og store havområder fjernet for å forbedre ytelsen av systemet.

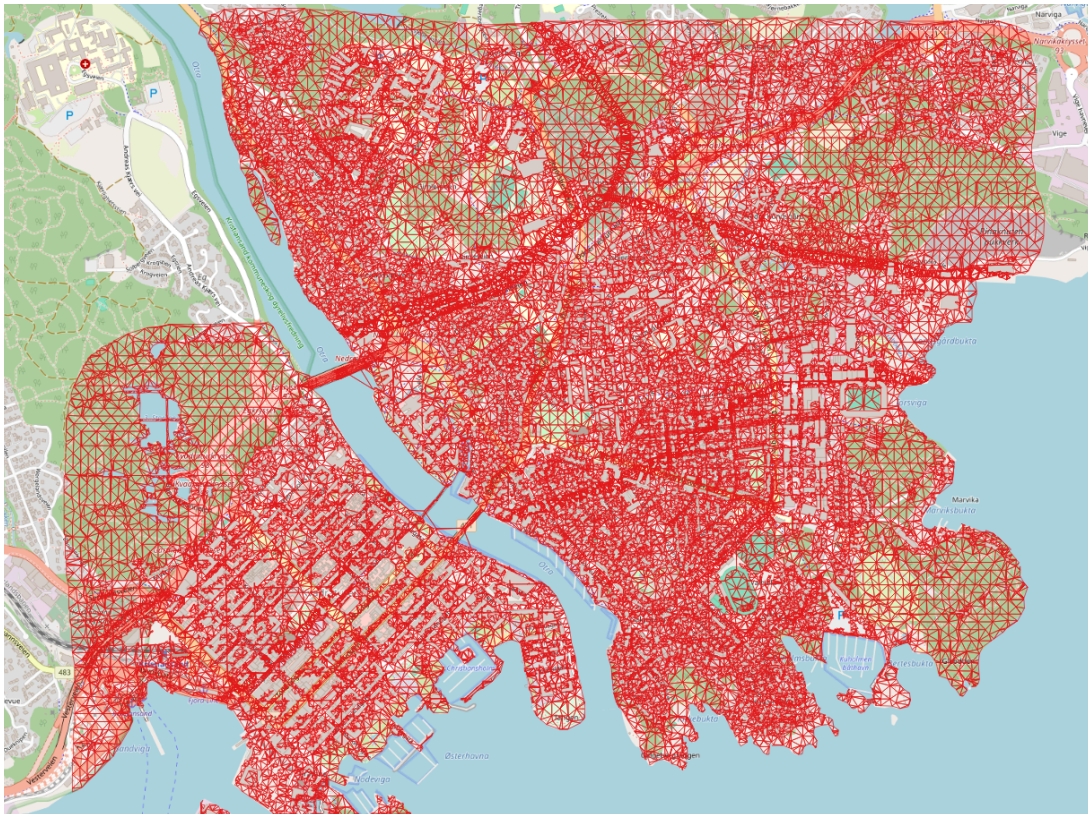


Figur 6: Alle bygninger markert av KI



Figur 7: Alle områder ikke markert av KI delt opp i mindre rektangler

Måten algoritmen for å finne kortest vei virker er å skape et linjenettverk hvor ingen av linjene krysser en bygning. Det markerte området ble derfor delt opp i flere mindre polygoner, for å skape et mer nøyaktig nettverk. For å kunne opprette linjene, ble det deretter satt et punkt i midten og hjørnene til hver polygon. Dette var for å kunne trekke linjer mellom punktene. Etterpå ble QGIS funksjonen "Symmetrical Difference" brukt for å fjerne alle linjer som krysser det originale markerte området. Resultatet, som vist i figur 8, tilsvarte linjenettverket som datagrunnlaget for algoritmen.



Figur 8: Linjer mellom alle punkt som ikke krysser umarkert område

3.2 Funksjoner

For å effektivt forklare noen kjernefunksjoner i applikasjonen brukes det en userstory til å sette funksjonene i en reell kontekst:

“Anders er helt alene og har fått en akutt alvorlig skade i armen. Han får ingen kontakt med noen av sykehusene og Anders vil derfor finne den korteste veien han kan løpe til sykehuset.”

Når Anders interagerer med løsningen så kalles det flere viktige funksjoner. Først velger han beredskapstjeneste som henter riktig data fra en tabell lagret i Supabase. Datahenting skjer via et Supabase kall til funksjonen “get_pointstest”. Som nevnt tidligere har dette kallet en innebygd begrensing, hvor den bare kan hente opp til 1000 objekter fra Supabase. Dette gjør at funksjonen er egnet til å hente data fra tabeller med opp til 1000 objekter. Større datahenting må derfor deles opp i flere kall, eller delegeres til backend-servere.


```

const fetchDataDirectly = async (tableName, type) => {
  try {
    console.log(`Henter data fra ${tableName}...`);
    const { data, error } = await supabase.rpc('get_pointstest', {
      table_name: tableName,
    });

    if (error) {
      throw new Error(`Feil ved henting av data fra ${tableName}: ${error.message}`);
    }
    // Behandle og returner data
  } catch (error) {
    console.error(`Feil ved henting av data fra ${tableName}:`, error);
    setError(`Kunne ikke hente data fra ${tableName}: ${error.message}`);
    return [];
  }
};

```

Figur 9: Henting av data fra databasen

Etter at han har hentet beredskapspunktene, vises de som markører på kartet. For hvert punkt beregnes avstanden fra brukerens posisjon, det nærmeste punktet identifiseres og en linje tegnes mellom bruker og dette punktet.

For at Anders skal finne alternativ rute må han markere ett start og ett slutt punkt. Det første som skjer når funksjonen blir kalt er at den lager et avgrensingsrektangel (*bounding box*). Rektangelet blir laget ved å sette de to punktene i hvert hjørne, og deretter forme et avgrensingsrektangel basert på dette. Posisjonen til de fire hjørnene i rektangelet vil deretter bli brukt som parametre til å hente kun linjer innenfor rektangelet. Linjene blir deretter hentet ut gjennom en spørring til databasen. Som nevnt tidligere i kapittelet over om kompleksitet i prosjektet, blir posisjonen til de markerte punktene transformert fra EPSG: 4326 til EPSG: 3395, for å samhandle med linjene i databasen.

```

app.route('/fetch_lines', methods=['POST'])
def fetch_lines():
  try:
    data = request.json
    min_x = data['min_x']
    min_y = data['min_y']
    max_x = data['max_x']
    max_y = data['max_y']

    # Hvor mye bounding box skal utvides
    expansion_factor = 1.2 # Øk med 20% hver iterasjon
    max_attempts = 5 # Maksimalt antall utvidelser
    attempt = 0

    while attempt < max_attempts:
      # Pagination variables
      limit = 20000 # Hvor mange linjer som skal hentes per spørring
      offset = 0 # Start på 0 for første spørring
      all_features = []

```

```

conn = get_db_connection()
cursor = conn.cursor()

while True:
    query = """
        SELECT id, ST_AsGeoJSON(geom)::text AS geometry, "POINTA", "POINTB", "POINTC"
        FROM "linesForShortestPath"
        WHERE ST_Intersects(
            geom,
            ST_Transform(
                ST_MakeEnvelope(%s, %s, %s, %s, 4326),
                3395
            )
        )
        LIMIT %s OFFSET %s
    """
    cursor.execute(query, (min_x, min_y, max_x, max_y, limit, offset))
    rows = cursor.fetchall()

    # Konverterer til GeoJSON
    features = []
    for row in rows:
        features.append({
            'type': 'Feature',
            'geometry': row[1],
            'properties': {
                'id': row[0],
                'pointA': row[2],
                'pointB': row[3],
                'pointC': row[4]
            }
        })

    all_features.extend(features)

    # Stopp hvis ingen flere linjer er funnet
    if len(features) < limit:
        break

    offset += limit

cursor.close()
conn.close()

if len(all_features) > 0:
    geojson = {
        'type': 'FeatureCollection',
        'features': all_features
    }
    return jsonify(geojson)

# Utvid boundingboxen hvis ingen linjer er funnet
print(f"Expanding bounding box (attempt {attempt + 1})...")
min_x -= (max_x - min_x) * (expansion_factor - 1) / 2
max_x += (max_x - min_x) * (expansion_factor - 1) / 2
min_y -= (max_y - min_y) * (expansion_factor - 1) / 2
max_y += (max_y - min_y) * (expansion_factor - 1) / 2
attempt += 1

except Exception as e:
    print("Error fetching lines:", str(e))
    return jsonify({'error': str(e)}), 500

```

Figur 10: Simplifisert `fetch_lines` funksjon for å vise logikk

Etter linjene er hentet fra databasen blir funksjonen for kortest rute kjørt. Denne funksjonen transformerer også punktene som ble satt av brukeren til EPSG: 3395. Funksjonen lager deretter et nettverk med linjene som er på innsiden av bound box, og kjører “shortestpathpointtopoint”. Dette er en innebygd algoritme fra QGIS som finner korteste rute fra et punkt til et annet. Hvis algoritmen ikke finner en gyldig rute mellom punktene, vil den forsøke å gå tilbake, og øke boksen med 20% fem ganger. Etterpå vil klienten prosessere resultatet og legge det inn på kartet som en linje. Etter at kartet har blitt oppdatert, kan Anders følge veien for å finne kortest vei til nærmeste sykehus.

```

@app.route('/shortestpath', methods=['POST'])
def shortest_path():
    try:
        data = request.json
        start_point = data['start_point']
        end_point = data['end_point']

        # Transformer punktene til EPSG: 3395
        formatted_start_point = reproject_point_to_epsg3395(start_point)
        formatted_end_point = reproject_point_to_epsg3395(end_point)

        # Hent linjene fra backend
        response = fetch_lines_from_backend(min_x, min_y, max_x, max_y)
        if not response['success']:
            return jsonify({'success': False, 'error': 'No lines found in the bounding box'})

        line_data = response.get('geojson', {})
        if not isinstance(line_data, dict):
            return jsonify({'success': False, 'error': 'Invalid GeoJSON data received'})

        # Lag et QGIS lag med linjene
        layer = QgsVectorLayer("LineString?crs=EPSG:3395", "network", "memory")
        provider = layer.dataProvider()
        for feature in line_data['features']:

            # Processor hver linje (simplifisert)
            qgs_feature = QgsFeature()
            geometry = feature['geometry']
            wkt = f"LINESTRING ({', '.join([f'{{x[0]}} {{x[1]}}' for x in geometry['coordinates']])})"
            qgs_feature.setGeometry(QgsGeometry.fromWkt(wkt))
            provider.addFeatures([qgs_feature])

        # Definer parametre for shortestpathpointtopoint
        params = {
            'INPUT': layer,
            'STRATEGY': 0,
            'DEFAULT_DIRECTION': 2, # Algoritmen kan gå frem og tilbake
            'TOLERANCE': 1, # Hvor mye mellomrom for at en linje skal koble seg til en annen
            'START_POINT': formatted_start_point,
            'END_POINT': formatted_end_point,
            'OUTPUT': 'TEMPORARY_OUTPUT'
        }

        # Kall algoritmen
        result = processing.run("native:shortestpathpointtopoint", params)
        output_layer = result['OUTPUT']

        # Hent resultat
        features = []
        for feature in output_layer.getFeatures():
            features.append({
                'geometry': feature.geometry().asWkt(),
                'attributes': feature.attributes()
            })

        return jsonify({'success': True, 'features': features})

    except Exception as e:
        return jsonify({'success': False, 'error': str(e)})

```

Figur 11: Simplifisert shortestpath funksjon for å vise logikk

4.0 Diskusjon

Modellen finner raskeste vei, men den tar ikke hensyn til flere viktige ting i virkeligheten, hvilket gjør at rutene den forslår ikke alltid er brukbare. Dette innebærer at det er flere viktige forhold som har direkte konsekvenser for hvor realistisk modellen er i praktisk bruk. Et sentralt problem med modellen er at den ikke tar høydeforskjeller i betraktning. Den opererer med et todimensjonalt kartbilde og tar dermed ikke stilling til topografiske forhold som bakker og store høydeforskjeller. Slike elementer kan i praksis gjøre enkelte ruter utilgjengelige eller mer krevende enn hva modellen forutser.

Videre tar modellen ikke hensyn til motorveier og hovedveier. Slike veier kan utgjøre en fysisk barriere siden det kan være farlig å krysse dem uten gangfelt eller broer. Når dette ikke reflekteres i modellen, oppstår det ruter som fremstår som korte, men også er urealistiske og farlige i praksis. Et annet viktig aspekt er fraværet av informasjon om vannmasser. Elver og kystlinjer kan representere en stor barriere i terrenget, og uten bro eller fergeforbindelser vil slike områder være vanskelige å krysse. Når dette blir ignorert, kan den beregne ruter som krysser rett over vann uten å ta hensyn til passasjerer.

Til slutt må det påpekes at modellen ikke vurderer tilgjengelighet i bred forstand. Terreng som er utilgjengelig for rullestolbrukere eller personer med bevegelseutfordringer, områder med trapper, gjerder eller stengsler, samt områder med begrenset ferdsel, er ikke representert i datagrunnlaget. Dette svekker modellens nytteverdi for brukere med spesielle behov. Selv om brukbarheten til modellen svekkes, gir det nyttig informasjon om hva som videre kan forbedres.

4.1 Beslutninger

Beslutningen om å ta i bruk et navigasjonsnett overfor andre alternativer slik som en RRT, ble gjort av to grunner. Primærårsaken til at det ble prioritert å bruke et navigasjonsnett relaterer til at ved bruken av et navigasjonsnett har man teknisk sett et sett med mange små veier som kobles sammen, dette kan brukes til å finjustere ruter som potensielt kan være farlige å gå, slik at man unngår å foreslå ruter som kan medføre fare på liv og helse.

Den andre årsaken relaterer til hastigheten en rute kan genereres på via et navigasjonsnett kontra et RRT. Dersom et navigasjonsonett allerede har blitt generert vil det være betydelig raskere å bruke dette enn en RRT. Da hensikten med systemet er at den skal være nyttig i krisesituasjoner, blir hastigheten den genererer ruter på være en hovedfaktor i hvor nyttig tjenesten er. I konteksten av et distribuert system antas det at denne genereringen allerede har blitt gjort. Hvilket vil gjøre navigasjonsnettet til det raskeste alternativet.

En annen viktig beslutning i konteksten av prosjektet går på prioriteringen av KI-komponenten i oppgaven ovenfor andre elementer som ville gjøre applikasjonen mer representativ for virkeligheten. Dette kommer av at topografiske kart allerede er et etablert felt, og derfor vil samfunnsinteressen av et studentprosjekt som fokuserer på dette være mindre enn en korrelativ arbeidsinnsats innen bruken av KI-bildegjenkjenning for å generere nye ruter. Samfunnsnyttien av dette ble en avgjørende faktor i beslutningen om å vinkle oppgaven mer mot et mulighetsstudie enn en mer virkelighetsrepresenterende applikasjon som kan stilling til utfordrende terreng.

4.2 Hva har vi lært

Vi har lært at klare ansvarsgrenser og bevisste tekniske valg er avgjørende for å lykkes med avansert GIS-behandling. Ved å dele opp arbeidet slik at backend håndterte preprosessering av over 110 000 data-punkter og generering av navigasjonsnett, mens frontend kunne fokusere på brukervennlig visualisering og interaksjon, unngikk vi ytelsesflaskehalser. Konsekvent bruk av projeksjonskonvertering mellom EPSG:4326 og EPSG:3395 viste seg å være en forutsetning for nøyaktige avstandsberegninger og pålitelige ruter, særlig over større områder. Valget av et forhåndsgenerert rutenett framfor RRT-algoritmen ga oss betydelig raskere beregninger og bedre kontroll over “farer” i terrenget, men lærte oss også at avhengighet til QGIS sin Python-tolk krever grundig dokumentasjon og vedlikehold.

Vi erfarte også hvor kritisk datakvalitet og validering er for brukertilfredshet og pålitelighet (Secoda, u.å). Manuell gjennomgang av nødetatspunkter fra GeoNorge og OverPassTurbo avdekket uoverensstemmelser som ellers ville medført til ineffektive ruter. Fysisk testing av algoritmens forslag bekreftet viktigheten av praktisk verifisering, og tilbakemeldinger fra

brukere understreket at tydelig, grafisk framstilling bygger tillit. Vi har lært at neste steg må inkludere høydeprofil- og barriereinformasjon (motorveier, gjerder, vannmasser) for å sikre at løsningen fungerer robust i virkelige redningsscenarioer.

5.0 Veien videre

Med mer tid og ressurser kunne prosjektet blitt videreutviklet på flere områder. En naturlig utvidelse ville vært implementering av topografiske kart, som kunne gi informasjon om høydeforskjeller og terrengtype. Dette ville gjort ruteberegningene mer realistiske og nyttige i situasjoner der fremkommelighet påvirkes av terrengets utforming. Det er også potensial for å forbedre nøyaktigheten i datagrunnlaget. Selv om datakildene er anerkjente og delvis kvalitetssikret, ble det observert enkelte mangler i gjenkjenningen av bygninger ved bruk av kunstig intelligens. Mer omfattende treningsdata eller tilpasning av data til lokale områder kunne bidratt til mer presise resultater. I tillegg kunne det vært verdifullt å gjennomføre litteraturstudier med fokus på navmesh-teknologi, for å undersøke hvordan navigasjonsnettverk kan benyttes i kartbaserte systemer. Slik innsikt kunne styrket metodene for å modellere alternative ruter i ustrukturert terreng. Disse tiltakene ville økt systemets nøyaktighet, robusthet og anvendelighet i beredskapssammenheng.

Referanser

Jurkowska, Justyna (16. Mars 2023) Geographic Coordinate Systems 101: A Primer for Software *Generalists*. 8thlight.

<https://8thlight.com/insights/geographic-coordinate-systems-101>

Kartverket. (2024, 11. april). *Brannstasjoner*. Geonorge.

<https://kartkatalog.geonorge.no/metadata/brannstasjoner/0ccce81d-a72e-46ca-8bd9-57b362376485>

Mostafa, A. (2016, 17. mai). *Server-side rendering of big data with GPUs*. Heavy.AI.

<https://www.heavy.ai/blog/gpu-rendering>

Nasjonal sikkerhetsmyndighet. (2022). *Norske datasenter og digital autonomi*. Hentet fra

<https://nsm.no/getfile.php/137781-1643362767/NSM/Filer/Dokumenter/Rapporter/Norske%20datasenter%20og%20digital%20autonomi.pdf>

Secoda. (n.d.). *Why is data quality important in real-time data processing?* Secoda.

<https://www.secoda.co/blog/why-is-data-quality-important-in-real-time-data-processing>

Supabase. (u.å.). *JavaScript: Upgrade guide*. Supabase Docs. Hentet 16. mai 2025, fra

<https://supabase.com/docs/reference/javascript/v1/upgrade-guide>

Utdanningsdirektoratet. (u.å.). *Veileder i beredskapsplanlegging*. Hentet 16. mai 2025, fra

<https://www.udir.no/kvalitet-og-kompetanse/sikkerhet-og-beredskap/veileder-i-beredskapsplanlegging/>